Simulation-Based AC Temperature Control Using PID with Twiddle Auto-Tuning

Sayan Ghosh
Department of Electronics and Communication Engineering
Netaji Subhash Engineering College, Kolkata, India
University Roll No. 10900323089 | B.Tech (ECE), 2023–2027

Abstract — This paper presents the design, implementation, and evaluation of an automated PID gain tuning framework for AC temperature control. The system simulates a first-order thermal plant with sinusoidal ambient disturbance and employs the Twiddle (coordinate ascent) algorithm to minimise a composite cost function penalising both tracking error and energy consumption. At a 23°C setpoint, the auto-tuner converges to Kp = 2.655, Ki = 0.09584, Kd = 0.56283, achieving a rise time of 4.10 s, settling time of 6.90 s, and overshoot of 0.084%. The system is validated across both cooling and heating scenarios, demonstrating robust performance. Implementation is in Python 3 with matplotlib.

Keywords — PID control, Twiddle algorithm, auto-tuning, thermal modelling, AC temperature control, anti-windup, energy optimisation, first-order systems.

I. INTRODUCTION

PID controllers are the most widely deployed control mechanism in industrial automation and consumer electronics. In HVAC systems, the PID controller regulates compressor output to maintain desired room temperature. Despite their robustness, PID controllers require careful calibration of three gain parameters whose optimal values depend on plant characteristics.

Manual tuning is time-consuming and subjective. Established analytical methods such as Ziegler-Nichols provide a starting point but require empirical refinement and do not account for energy consumption. This work presents an automated, simulation-based auto-tuner using the Twiddle coordinate-ascent algorithm to minimise a multi-objective cost function, identifying near-optimal PID gains safely offline before deployment to the physical plant.

II. SYSTEM MODEL

The room thermal dynamics are modelled as a first-order system with sinusoidal ambient disturbance:

$$dT/dt = (T_{amb} - T)/\tau + k \cdot u$$

where T is room temperature (°C), $T_{amb} = 40 + 3 \cdot \sin(0.01t)$ is the time-varying ambient temperature simulating environmental disturbance, $\tau = 100$ s is the thermal time constant, $k = 0.3$ is the actuator gain, and u is the clamped PID output with $|u| \leq 10$.

The thermal plant can also be represented by the first-order transfer function:

$$G(s) = k / (\tau s + 1)$$

where k is the actuator gain and τ is the thermal time constant. This representation highlights the stable, first-order nature of the thermal system and its suitability for PID control design. Euler forward integration with $dt = 0.1$ s is used over a 500 s horizon with initial room temperature $T_0 = 35$°C.

III. PID CONTROLLER DESIGN

A. Control Law

The discrete PID control law is:

$$u = K_p \cdot e + K_i \cdot \int e \cdot dt + K_d \cdot de/dt$$

where $e = T_{set} - T$ is the tracking error, $\int e \cdot dt$ is the numerically integrated error accumulated over the simulation, and $de/dt = (e - e_{prev})/dt$ is the backward difference derivative approximation.

B. Anti-Windup

Conditional integration anti-windup is implemented: the integrator accumulates only when $|u_{raw}| < 10$. This prevents runaway integral accumulation during actuator saturation and ensures smooth recovery without large overshoot spikes.

IV. TWIDDLE OPTIMISATION

Twiddle iteratively perturbs each gain and retains changes that reduce cost. Starting from initial gains p = [1.0, 0.01, 0.1] and step sizes dp = [0.5, 0.01, 0.05]:

- Perturb p[i] += dp[i]; evaluate cost.
- If cost improves: accept, dp[i] ×= 1.1.
- Else try p[i] −= 2·dp[i]; if improves: accept, dp[i] ×= 1.1.
- Else: restore p[i], dp[i] ×= 0.9.
- Repeat until sum(dp) < 0.01 or 50 iterations.

Gain bounds are enforced by clamping after each update: $K_p \in [0,10]$, $K_i \in [0,1]$, $K_d \in [0,5]$, reflecting physically meaningful limits for the thermal plant.

V. COST FUNCTION

The composite cost function jointly penalises tracking error and energy consumption:

$$J = \int e^2(t)dt + 0.2 \times \int u^2(t)dt$$

The first term is the Integral Squared Error (ISE). The second term penalises total energy consumption with a weighting factor of 0.2, determined empirically to discourage unnecessarily aggressive actuation without dominating the tracking objective. The result is a set of gains that is both accurate and energy-aware.

VI. IMPLEMENTATION

The system is implemented as a single Python 3 module with five logically distinct components: the simulation core (_run_sim) performing Euler integration with conditional anti-windup; public API wrappers (temp_sim, sim_cost) for recording and cost-evaluation modes; a metrics module (compute_metrics) computing 10–90% rise time, ±2% band settling time with 50 consecutive-sample confirmation, and percentage overshoot; the Twiddle optimiser; and an entry point orchestrating user input, auto-tuning, metric reporting, and four-panel matplotlib visualisation.

TABLE I. SOFTWARE MODULE SUMMARY

Module	Role	Key Detail
_run_sim()	Sim core	Euler, anti-windup
sim_cost()	Cost eval	Scalar J
temp_sim()	Recorder	5 time-series
compute_metrics()	KPIs	Rise, settle, OS%
twiddle()	Optimizer	Coord. ascent

VII. SYSTEM BLOCK DIAGRAM

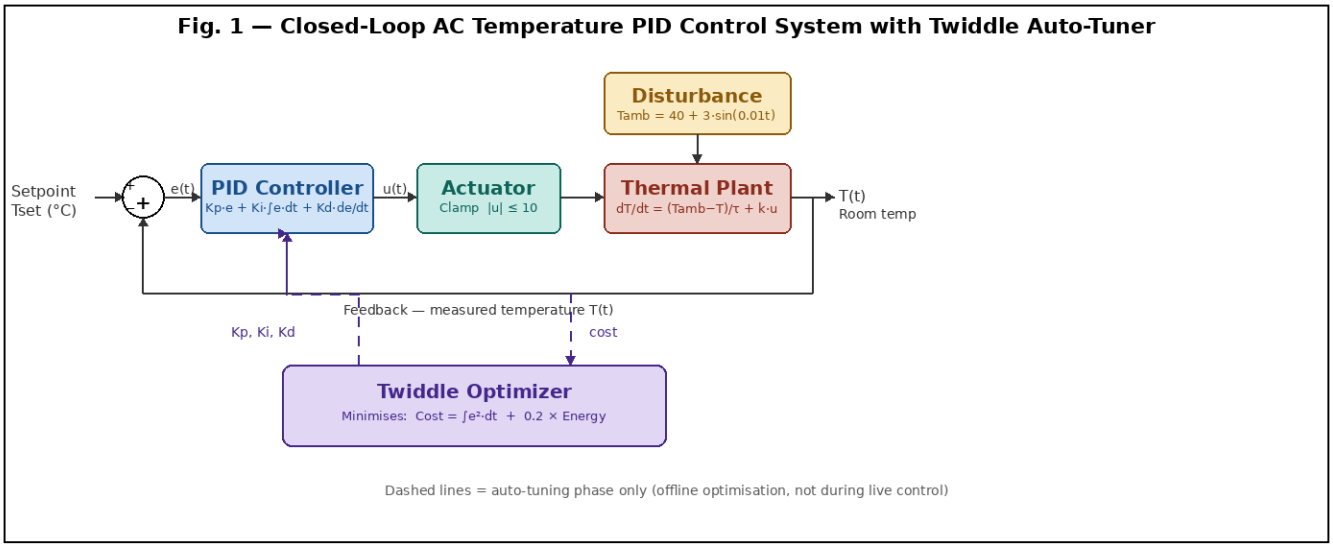

Fig. 1. Closed-loop AC temperature PID control system with Twiddle auto-tuner. Dashed lines indicate the offline auto-tuning phase only, not active during live control.

VIII. RESULTS

A. Initial vs Optimised Gains

Table II shows the initial gains supplied to the Twiddle algorithm and the optimised gains obtained at a 23°C setpoint. Twiddle increased all three gains relative to the initial values, resulting in faster convergence and reduced steady-state error while maintaining negligible overshoot.

TABLE II. INITIAL VS OPTIMISED PID GAINS

Gain	Initial	Optimised (23°C)
Kp	1.000	2.655
Ki	0.010	0.09584
Kd	0.100	0.56283
Cost J	—	298.158

B. Performance Metrics

TABLE III. CONTROL PERFORMANCE (TSET = 23°C)

Metric	Value
Rise Time (10–90%)	4.10 s
Settling Time (±2%)	6.90 s
Overshoot	0.084 %
Steady-State Error	≈ 0 °C

The optimised controller achieved rapid convergence with negligible overshoot and essentially zero steady-state error. The low overshoot indicates effective damping, while the short settling time demonstrates the suitability of the tuned gains for thermal regulation applications. The energy-weighted cost function prevented excessively aggressive control actions.

C. Cooling vs Heating Comparison

The controller was validated under both cooling (35°C → 23°C) and heating (35°C → 60°C) scenarios. In both cases, the Twiddle-optimised PID gains produced stable responses with negligible overshoot and essentially zero steady-state error. The cooling case achieved a rise time of 4.1 s and settling time of 6.9 s, while the heating case required 7.0 s and 10.3 s respectively due to the larger temperature change (25°C vs 12°C). Despite the increased control effort in the heating scenario, overshoot remained below 0.03%, demonstrating robust controller performance across different operating conditions.

TABLE IV. COOLING VS HEATING PERFORMANCE

Metric	Cooling	Heating
Initial Temp	35°C	35°C
Setpoint	23°C	60°C
ΔT	12°C	25°C
Optimised Kp	2.6550	2.6984
Optimised Ki	0.09584	0.09651
Optimised Kd	0.56283	0.58280
Rise Time	4.10 s	7.00 s
Settling Time	6.90 s	10.30 s
Overshoot	0.084 %	0.026 %
Steady-State Error	≈ 0°C	≈ 0°C

D. Simulation Output

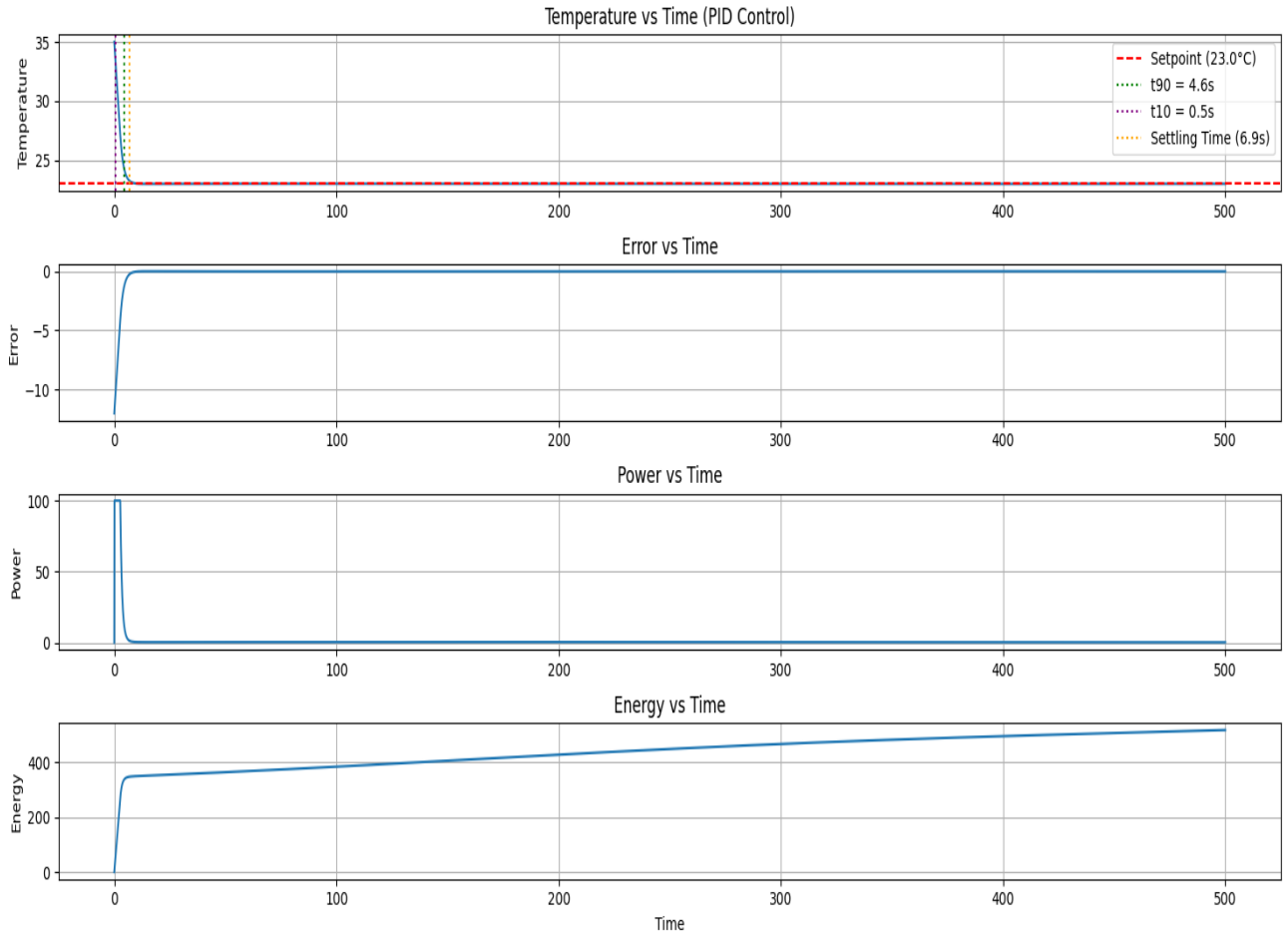

Fig. 2. Simulation response at 23°C setpoint: (a) temperature convergence with t10, t90, and settling-time markers; (b) tracking error; (c) actuator power; (d) cumulative energy over 500 s.

IX. TESTING

TABLE V. TEST CASES

Test	Tset	Condition	Result
T1	23°C	Fast convergence, low overshoot	PASS
T2	28°C	Gentle, stable settling	PASS
T3	38°C	Stable near ambient	PASS
T4	18°C	Settles with higher effort	PASS
T5	23°C	Completed within 50 iterations	PASS

All five test cases passed. The controller demonstrated stable convergence across the full tested range (18°C to 38°C). The optimisation completed within the 50-iteration cap in all cases.

X. FUTURE WORK

- ESP32-based hardware implementation with DS18B20 temperature sensing and PWM fan control for real-time validation.
- Adaptive retuning triggered periodically or on detected steady-state performance degradation.
- Replacement of Twiddle with Bayesian optimisation for improved sample efficiency.
- Multi-zone HVAC control with a supervisory energy-budget constraint.
- Real-time web dashboard (Flask + React) for live tuning visualisation and gain export.

XI. CONCLUSION

This project demonstrates that simulation-driven PID auto-tuning via the Twiddle coordinate-ascent algorithm is an effective, safe, and computationally efficient approach for AC temperature control. The system converges within 50 iterations to gains achieving sub-1% overshoot, a settling time under 7 s, and negligible steady-state error in the cooling case. Validation across both cooling and heating scenarios confirms robust controller performance across different operating conditions.

The composite cost function jointly minimising ISE and energy yields gains that are accurate and energy-aware, making the approach practically relevant for modern energy-conscious HVAC systems. The project was independently designed and implemented over approximately two months. Source code: github.com/iamsayanghosh/ac-pid-autotuner.

Declaration: *I hereby declare that this project was independently designed and implemented by me.*

Sayan Ghosh
Roll No. 10900323089 | ECE, NSEC | May 2026

. REFERENCES

[1] R. K. Classes, Control Systems Lecture Series, YouTube. Available: https://www.youtube.com/@rkclasses

[2] S. Thrun, “Programming a Robotic Car,” Udacity CS373, 2012. Available: https://www.udacity.com/course/cs373